SMARTFLOW

Technical Report — Multi-Object Tracking & Counting From Video

1. Project Objective

SMARTFLOW is a computer-vision traffic monitoring system that detects vehicles in a video, tracks them across frames, and counts each tracked vehicle once when it crosses a predefined counting line. The project demonstrates an end-to-end multi-object tracking and counting pipeline with an interactive Streamlit dashboard.

2. Problem and Approach

Frame-by-frame detection can detect the same vehicle repeatedly and therefore produce incorrect counts. SMARTFLOW combines object detection with multi-object tracking. Each detected vehicle receives a tracking ID, allowing its movement to be followed across consecutive frames. A virtual line is then used as the counting boundary.

The submitted test video is 1920 × 1080, 25 FPS, and contains 525 frames (approximately 21 seconds). A vertical counting line is positioned at approximately 60% of the video width. The system compares the previous and current horizontal center position of each tracked vehicle to determine whether it crossed the line.

3. Detector Choice — YOLO11

SMARTFLOW uses YOLO11 through the Ultralytics framework. YOLO was selected because it is an open-source object detector suitable for fast video inference. A pretrained model was used, which avoids the additional training and dataset preparation required for a custom detector.

The application focuses on vehicle-related classes: car, motorcycle, bus, and truck. The pretrained COCO taxonomy does not provide a dedicated auto-rickshaw class, so some region-specific vehicles may be assigned to a visually similar class. This is documented as a limitation.

4. Tracker Choice — ByteTrack

ByteTrack is used for multi-object tracking. It maintains object identities between frames, which is essential for counting unique vehicles instead of counting every detection. The implementation uses persistent tracking IDs and stores the previous position of each tracked object.

When a tracked object's center moves from one side of the counting line to the other, a crossing event is generated. The direction is recorded as a left or right crossing relative to the virtual line.

5. Counting and Double-Count Prevention

SMARTFLOW uses line-crossing logic. For every tracked vehicle, the previous horizontal center coordinate is stored. A crossing is detected when the previous coordinate and current coordinate lie on opposite sides of the configured vertical line.

After an ID is counted, it is stored in a counted-ID set. That ID cannot generate another count, preventing repeated counting while the same vehicle remains visible or moves around the boundary. Left and right crossings are maintained as cumulative directional counts.

6. Dashboard and Data Logging

The Streamlit dashboard displays the running vehicle count, cumulative left and right crossings, current vehicle composition, traffic density, flow estimate, annotated tracking video, analytics, and manual validation. OpenCV handles video processing and annotation, while Pandas is used for CSV data handling.

Traffic observations are saved in reports/traffic_data.csv. The manual comparison is saved in reports/count_report.csv.

7. Experimental Results and Manual Validation

Metric	Result
Input video	traffic.mp4
Resolution / FPS	1920 × 1080 / 25 FPS
Frames / duration	525 / approximately 21 seconds
System total	8
System left / right	4 / 4
Manual total	9
Manual left / right	4 / 5
Absolute difference	1
Count accuracy	88.89%

Manual review of the same video recorded 9 crossings: 4 left and 5 right. SMARTFLOW recorded 8 crossings: 4 left and 4 right. The absolute difference is 1 vehicle, producing a reported count accuracy of 88.89%. The validation file is reports/count_report.csv and is marked VERIFIED.

8. Known Failure Cases and Robustness

Tracking can degrade when vehicles are heavily occluded, overlap for extended periods, or temporarily disappear from detection. Such events can cause an ID to be lost or reassigned and may affect counting accuracy, particularly near the counting line.

The pretrained detector may also classify region-specific vehicle types imperfectly. Density is a simple heuristic based on the number of vehicles visible in a frame and is not a calibrated traffic-engineering measurement. Flow rate is a baseline estimate derived from crossing events and elapsed time.

9. Reproducibility

The project is organized into source modules for detection, tracking, counting, direction analysis, density estimation, data logging, and supporting analysis. The Streamlit dashboard is implemented in app.py and manual validation is implemented in manual_count_validator.py.

Run sequence:
1. Create and activate the Python virtual environment.
2. Install dependencies with **pip install -r requirements.txt**.
3. Place the video at **input/traffic.mp4**.
4. Run **python src/main.py**.
5. Run **streamlit run app.py**.
6. Run **python manual_count_validator.py** for manual validation.

10. Technology Stack

Python — application development
YOLO11 / Ultralytics — vehicle detection
ByteTrack — multi-object tracking
OpenCV — video processing and annotation
Pandas — traffic data handling
Streamlit — dashboard and demonstration

11. Project Pipeline

Video Input → YOLO11 Detection → ByteTrack Tracking → Persistent Object IDs → Line-Crossing Detection → Unique Vehicle Count → CSV Logging → Streamlit Dashboard

12. Conclusion

SMARTFLOW demonstrates an end-to-end multi-object tracking and counting solution using an open-source computer-vision stack. It detects vehicles, assigns tracking IDs, follows them across frames, counts unique line crossings, records directional crossings, logs traffic information, and presents the results through an interactive dashboard.

The system satisfies the core objective of detecting, tracking, and counting moving objects without repeatedly counting the same tracked object. Manual validation produced an 88.89% count accuracy. Future improvements could include detector fine-tuning for local vehicle classes, better calibration of the counting line and density thresholds, and improved robustness under severe occlusion and re-entry conditions.